

Course Outline: Background Job Triggers

Title: Background Job Triggers: What Starts a Background Job **Skill:** Skill 45: Background processing
Learnpack: + Tipos de disparadores de procesamiento en segundo plano **File:** s45-w15d45-tipos-de-disparadores-de-procesamiento-e **Prerequisite:** s45-w15d45-introduccion-procesamiento-en-segundo-pl **Target Audience:** Beginner developers who understand what background processing is and now need to recognize when and how background jobs are initiated **Total Lessons:** 9 **Format:** Conceptual (0% hands-on)

Course Description

A background job doesn't run on its own — something has to start it. That "something" is a trigger. This course maps the full taxonomy of background job triggers: user actions that kick off long-running work, telemetry and machine events that respond to system conditions, scheduled cronjobs that fire on a timer, third-party webhooks that arrive from external services, and chained jobs that one background task hands off to the next. Understanding this taxonomy helps you choose the right architecture for every asynchronous task you build.

Course Philosophy

This course emphasizes:

- Knowing the full trigger menu before designing any background processing feature
 - Recognizing each trigger type by its initiating condition (who or what causes the job to start)
 - Matching the trigger type to the architectural pattern — not all triggers use the same mechanism
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - What Is a Trigger	2
02 - Trigger Types	4
03 - Assessment	1
04 - Outro	1
Total	9

00.0 What Starts a Background Job 

Learning Objectives:

- Recognize that every background job has an initiating event — a trigger
- Understand why knowing the trigger type matters for architecture decisions
- Preview the five trigger categories covered in this course

Content Outline:

- Background jobs don't self-initiate — they are always started by an event
- The event can be: a user action, a machine condition, a clock, an external system, or another job
- Why trigger type matters: different trigger types are implemented differently and have different reliability guarantees
- Preview of the five categories:
 1. User-triggered: a human action causes the job
 2. Machine/telemetry-triggered: a system condition causes the job
 3. Cronjob-triggered: a timer causes the job on a schedule
 4. Third-party-triggered: an external service sends an event
 5. Chained: a completed job triggers the next job
- Course structure: Section 01 introduces the trigger concept and user-triggered processing; Section 02 covers the remaining four types and the decision guide

Transitions: This lesson frames the taxonomy. Section 01 defines what a trigger is and introduces the most common type.

Key Principles:

- Every background job has exactly one trigger — identifying it is the first design decision
- Trigger type determines the job's entry point, not what the job does
- The same task (e.g., generate a report) can be triggered by different trigger types

Examples:

- User clicks "Export CSV" → user-triggered export job starts
- Error rate crosses a threshold → telemetry-triggered alert job starts
- Every Monday at 9am → cronjob-triggered weekly summary starts
- Stripe sends a payment.completed webhook → third-party-triggered fulfillment job starts
- Image upload processing completes → chains into thumbnail generation job

01.0 What Is a Trigger

Learning Objectives:

- Define a trigger as the event that initiates a background job
- Explain the relationship between a trigger and a background job
- Distinguish between the trigger (what caused the job) and the job itself (what the job does)

Content Outline:

- Definition: a trigger is any event that causes a background job to be enqueued or executed
- The trigger is the cause; the job is the effect — they are linked but independent

- Triggers exist at the boundary: the trigger is in the "request layer" or "event layer"; the job runs in the "background layer"
- What a trigger contains: the signal that something needs to happen, and enough context to describe what
 - User-triggered: the user's intent + their identity + the relevant data
 - Machine-triggered: the condition that was detected + the system state at that moment
 - Schedule-triggered: the scheduled time + which job to run
- Triggers are not always explicit API calls — they can be messages on a queue, database row insertions, or webhook payloads
- Section preview: 01.1 covers user-triggered processing, the most common trigger type; Section 02 covers the remaining four

Transitions: This lesson defines the concept. 01.1 shows the first and most familiar trigger type in action.

Key Principles:

- A trigger and a job are decoupled by design — the trigger doesn't wait for the job to finish
- The trigger carries the minimum context the job needs to execute independently
- Triggers are ephemeral — the job must persist the context it needs, not rely on the trigger remaining available

Examples:

- Explicit trigger: `POST /export` → handler creates export job → job executes async
- Implicit trigger: user completes onboarding step → database flag changes → observer detects change → job enqueued
- Time trigger: cron fires at 00:00 UTC → job ID + timestamp sent to worker → job executes

01.1 User-Triggered Processing

Learning Objectives:

- Define user-triggered processing as background work initiated by a direct user action
- Identify common use cases where user actions should trigger background jobs
- Explain why user-triggered tasks often require immediate acknowledgment but delayed execution

Content Outline:

- User-triggered processing: the user explicitly requests something that requires background work
- The pattern: user action → handler acknowledges → job enqueued → job executes → result available
- Why not execute synchronously: the task takes too long, involves external services, or produces a resource for later retrieval
- Common use cases:
 - Exporting data (CSV, PDF, report) → user clicks "Export" → gets "processing" status → retrieves file when ready

- Sending bulk messages (email campaigns, notifications) → user clicks "Send" → gets acknowledgment → system sends asynchronously
- Uploading and processing media → user uploads image/video → gets "processing" → asset available after transformation
- Running a long computation → user triggers analysis → gets job ID → polls for result
- What the handler should return: a job ID or status URL that the user can poll or be notified about
- Anti-pattern: running the task synchronously and making the user wait — this creates poor UX and timeout risk

Transitions: User-triggered is the most intuitive trigger type. Section 02 introduces the trigger types that don't require a human to initiate them.

Key Principles:

- The user should never wait for a background job to complete — the handler returns immediately with a job reference
- User-triggered jobs are the most latency-sensitive: they need to be enqueued quickly so the user can see the "processing" state fast
- The job result must be retrievable by the user at a later time (polling, webhook callback, or push notification)

Examples:

- Export job: `POST /reports/export` → `{job_id: "j-123", status: "queued"}` → user polls `GET /jobs/j-123` until `{status: "complete", url: "..."}"`
- Bulk send: `POST /campaigns/42/send` → `{status: "sending", count: 1500}` → 1500 emails sent over next 10 minutes
- Image upload: `POST /photos` with file → `{photo_id: "p-99", status: "processing"}` → processed image ready at `GET /photos/p-99`

02.0 Other Trigger Types

Learning Objectives:

- Name the four remaining trigger types (telemetry/machine, cronjob, third-party, chained)
- Explain the key distinction between user-triggered and non-user-triggered processing
- Understand that Section 02 covers one trigger type per lesson plus the decision guide

Content Outline:

- User-triggered is initiated by a human; the other four trigger types are initiated by systems
- This distinction matters for design: non-user-triggered jobs don't have a user waiting for a response — they must be observable through logging and monitoring instead
- The four remaining types at a glance:
 - Machine/telemetry-triggered: the system itself detects a condition and fires the job
 - Cronjob-triggered: a clock fires the job on a predictable schedule
 - Third-party-triggered: an external service notifies your system, which fires the job
 - Chained processing: a completed job fires the next job in a pipeline

- Why categorize these separately: each has a different reliability profile, different failure mode, and different implementation approach
- Section preview: 02.1 covers machine and scheduled triggers; 02.2 covers third-party and chained triggers; 02.3 is the decision guide

Transitions: This overview sets up the next three lessons. 02.1 starts with the system-initiated trigger types.

Key Principles:

- Non-user-triggered jobs have no user to report status to — observability (from the previous learnpack) becomes critical
- The trigger type determines the entry point; it doesn't constrain what the job does
- Some trigger types are easier to test (user-triggered, scheduled) than others (third-party webhooks, chained)

Examples:

- Same job, different triggers: "generate daily usage report" can be scheduled (every day at midnight) OR machine-triggered (when the last user logs out) OR user-triggered (when an admin clicks "Run Now") — the trigger is separate from the job

02.1 Machine and Scheduled Triggers

Learning Objectives:

- Define machine/telemetry-triggered processing as jobs initiated by detected system conditions
- Define cronjob-triggered processing as jobs initiated by a time schedule
- Explain when each type is appropriate

Content Outline:

- **Machine/Telemetry-Triggered Processing:**
 - The system itself detects a condition and enqueues a job in response
 - The "trigger" is a threshold, event, or state change — not a human action
 - How it works: monitoring/telemetry system observes a condition → condition met → job enqueued
 - Common use cases:
 - Error rate exceeds 5% → alert job fires → sends notification to on-call team
 - Memory usage crosses 90% → cleanup job fires → frees cached resources
 - New data arrives in a stream → processing job fires → transforms and stores the data
 - User reaches inactivity threshold → re-engagement job fires → sends reminder email
 - Why it's useful: automates responses to system conditions without human polling
 - Failure mode: condition detection can produce bursts — many jobs fire simultaneously if many machines cross the threshold at the same time
- **Cronjob-Triggered Processing:**
 - A scheduler fires a job at a predetermined time — independent of any external event

- The trigger is purely temporal: "every day at 9am", "every hour", "first Monday of the month"
- How it works: cron scheduler checks the current time → matches a schedule → fires the job
- Common use cases:
 - Nightly data aggregation: aggregate today's events into summary records at midnight
 - Weekly email digest: send weekly newsletter every Monday morning
 - Hourly cache refresh: recompute cached report data every hour
 - Monthly billing: generate invoices on the first of every month
- Why it's useful: predictable timing makes it easy to reason about when work happens
- Failure mode: if the job takes longer than the schedule interval, jobs can overlap (the previous job hasn't finished when the next fires) — this is the "overlapping cronjob" problem covered in the next learnpack

Transitions: System-initiated triggers (machine and scheduled) don't require a human to fire them. 02.2 covers externally-initiated triggers and chained processing.

Key Principles:

- Machine-triggered is event-driven; cronjob-triggered is time-driven — both are system-initiated
- Machine-triggered jobs respond to conditions; they must be idempotent because the condition may trigger multiple times
- Cronjob-triggered jobs have predictable timing but must handle the case where the previous run is still in progress

Examples:

- Machine-triggered: telemetry shows API latency > 500ms for 5 consecutive requests → job enqueued to scale up the API tier
- Cronjob-triggered: `0 9 * * 1` (every Monday at 9am) → weekly usage report generated and emailed to team

02.2 Third-Party and Chained Triggers

Learning Objectives:

- Define third-party-triggered processing as jobs initiated by external service events (webhooks)
- Define chained processing as jobs initiated by the completion of another job
- Explain the reliability considerations specific to each type

Content Outline:

- **Third-Party-Triggered Processing (Webhooks):**
 - An external service sends a notification to your system, which fires a background job in response
 - The trigger is a webhook — an HTTP POST from the external service to your webhook endpoint

- How it works: external event occurs (e.g., payment completed) → external service calls `POST /webhooks/stripe` → your endpoint receives payload → job enqueued immediately → endpoint returns 200 fast
- Why return 200 fast: the external service expects a quick response; if you take too long, it may retry the webhook (which means your handler must be idempotent — see the previous learnpack)
- Common use cases:
 - Payment gateway webhook → fulfillment job fires → order processed, email sent
 - GitHub webhook on push → CI/CD job fires → build and tests run
 - Stripe subscription.cancelled → churn processing job fires → downgrade user's plan
 - Twilio SMS received → message processing job fires → chatbot response sent
- Reliability consideration: webhook delivery is not guaranteed — the external service will retry if you return a non-200 status or don't respond in time; your handler and job must be idempotent
- **Chained Processing (Internal Workflow):**
 - A completed background job enqueues the next background job — creating a pipeline
 - The trigger is the completion (or failure) of a previous job
 - How it works: Job A runs → on success, Job A enqueues Job B → Job B runs → on success, Job B enqueues Job C → etc.
 - Common use cases:
 - Image upload → resize job → thumbnail generation job → CDN upload job
 - Order placed → payment validation job → inventory deduction job → fulfillment job → notification job
 - Data ingestion pipeline: ingest → transform → validate → store → notify
 - Why chain instead of one big job: each step can be retried independently; failure in step 3 doesn't require restarting steps 1 and 2
 - Reliability consideration: long chains amplify failures — if step 5 fails repeatedly, the entire downstream pipeline stalls; each step must have clear success/failure criteria and retry limits

Transitions: You now know all five trigger types. 02.3 provides the decision guide for choosing between them.

Key Principles:

- Webhook endpoints must return 200 quickly and enqueue the job; never process the full job synchronously inside the webhook handler
- Chained processing adds resilience by making each step independently restartable
- Both third-party and chained triggers benefit greatly from idempotency — both can deliver the same trigger more than once

Examples:

- Webhook: Stripe sends `payment.succeeded` → handler stores raw payload and enqueues payment-processing job → returns 200 in < 100ms
 - Chained: video upload → transcoding job runs → on success enqueues thumbnail-extraction → on success enqueues metadata-indexing → on success enqueues CDN-publish
-

02.3 Choosing the Right Trigger

Learning Objectives:

- Apply a decision framework to choose the appropriate trigger type for a given scenario
- Recognize the signals that indicate each trigger type
- Avoid common mismatches between scenario and trigger type

Content Outline:

- The decision question: "What causes this job to need to run?"
 - A human action → user-triggered
 - A system condition or data event → machine/telemetry-triggered
 - A specific time or schedule → cronjob-triggered
 - A message from an external service → third-party-triggered
 - A previous job completing → chained processing

- Decision framework in table form:

Signal	Trigger Type
User clicks a button or submits a form	User-triggered
A metric crosses a threshold	Machine/telemetry-triggered
New data arrives in a stream	Machine/telemetry-triggered
"Run this every N minutes/hours/days"	Cronjob-triggered
An external service sends a webhook	Third-party-triggered
Previous job succeeded	Chained processing
A complex multi-step pipeline	Chained processing

- Common mismatches and why they fail:
 - Using a cronjob to poll for user actions: wastes compute, adds latency; use user-triggered instead
 - Running webhook processing synchronously inside the endpoint handler: risks timeout and duplicate processing; enqueue a job immediately and return 200
 - One massive job for a multi-step pipeline: any step failure requires full restart; use chained processing instead
 - Machine-triggered job without idempotency: the condition may fire repeatedly; jobs run multiple times with side effects
- The decision is not always clear-cut: some scenarios are naturally multi-trigger (e.g., a report that can be triggered by a user OR by a schedule) — in that case, the job logic is the same; only the entry point changes

Transitions: The decision framework closes the taxonomy. The assessment tests application of this framework to new scenarios.

Key Principles:

- One job can have multiple valid trigger types — the trigger is the entry point, not the job identity
- When in doubt: if a human must manually initiate it → user-triggered; if no human is involved → one of the four system triggers
- The trigger type affects observability requirements: user-triggered jobs need a job status the user can check; non-user-triggered jobs need logging and monitoring the team can check

Examples:

- "Send a welcome email when a new user signs up" → user-triggered (signup action)
 - "Archive old records once a week" → cronjob-triggered
 - "Detect and flag fraudulent transactions in real time" → machine/telemetry-triggered (anomaly detected)
 - "Process a payment confirmation" → third-party-triggered (payment gateway webhook)
 - "Generate a thumbnail after image processing" → chained (triggered by image processing job completion)
-

03.0 Assessment 🧠

Learning Objectives:

- Identify the correct trigger type for each scenario
- Apply the decision framework to new, unfamiliar scenarios
- Recognize mismatches between scenarios and incorrectly chosen trigger types

Question Format: Scenario-based (multiple-choice)

Questions:

1. A user clicks "Download Report" in your app. The report takes 45 seconds to generate. What trigger type should start the generation job?
 - A) Cronjob-triggered
 - B) Third-party-triggered
 - C) User-triggered ✓
 - D) Chained processing
2. Your app needs to generate a weekly summary email and send it to all users every Monday at 8am. What trigger type is this?
 - A) User-triggered
 - B) Machine/telemetry-triggered
 - C) Chained processing
 - D) Cronjob-triggered ✓

3. Your API's error rate exceeds 10% for 2 consecutive minutes. You want a job to fire automatically and alert your on-call team. What trigger type is this?
- A) Cronjob-triggered
 - B) Machine/telemetry-triggered ✓
 - C) User-triggered
 - D) Third-party-triggered
4. Stripe sends a `payment.succeeded` event to your `/webhooks/stripe` endpoint. A job needs to process the payment, update the order, and send a confirmation email. What trigger type is this?
- A) User-triggered
 - B) Cronjob-triggered
 - C) Third-party-triggered ✓
 - D) Machine/telemetry-triggered
5. A user uploads a photo. After the upload is stored, a resize job runs. When the resize job completes, a thumbnail job starts. When the thumbnail job completes, a CDN upload job starts. What trigger type connects these jobs?
- A) User-triggered (the initial upload)
 - B) Chained processing ✓
 - C) Cronjob-triggered
 - D) Third-party-triggered
6. Your webhook endpoint receives a `payment.succeeded` event from Stripe. The developer processes the full payment synchronously inside the webhook handler (it takes 8 seconds). What problem does this create?
- A) No problem — synchronous processing is fine in webhook handlers
 - B) Stripe may retry the webhook thinking delivery failed, causing duplicate processing ✓
 - C) The job will never run because webhooks are one-time events
 - D) The endpoint will return a 400 error automatically
7. Your app has a long pipeline: ingest data → transform → validate → store. You implement this as one single large background job. A failure in the validation step requires restarting the entire pipeline. What trigger type pattern would better address this?
- A) Cronjob-triggered
 - B) Machine/telemetry-triggered
 - C) Chained processing ✓
 - D) User-triggered
8. A developer is using a cronjob that fires every 5 minutes to check if any users have completed the onboarding step, then fires the welcome email job. What would be a better trigger type for this?
- A) Third-party-triggered
 - B) Cronjob-triggered (it's already fine as-is)

- C) Machine/telemetry-triggered (or user-triggered when onboarding completes) ✓
- D) Chained processing

Evaluation Criteria:

- Correctly maps scenarios to trigger types using the decision framework
- Identifies mismatches between scenario and trigger type
- Recognizes the reliability implications of each trigger type

Score Interpretation:

- 8/8: Trigger types are second nature — you can design any background processing architecture
 - 6–7: Review any missed scenarios and re-read the relevant lesson
 - 4–5: Re-read Section 02 with focus on the decision framework
 - Below 4: Restart from Section 01
-

04.0 Triggers in Your System

Content Outline:

- Course recap: what students accomplished
 - Named the five trigger types: user, machine/telemetry, cronjob, third-party, chained
 - Understood the key distinction: user-initiated vs system-initiated
 - Applied the decision framework: "what causes this job to need to run?"
 - Recognized common mismatches and their failure modes
- Connection to bigger picture: every background processing system you build will use one or more of these trigger types. Recognizing the trigger type is the first step in every design decision: which queue to use, how to handle retries, what observability you need, and whether idempotency is required
- Next steps: the next two learnpacks go deeper into two of these trigger types in practice — cronjobs (how to configure, observe, and handle overlapping) and queues (the mechanism that enables user-triggered, third-party, and chained processing at scale)
- Resources for further exploration:
 - AWS EventBridge documentation — a production event bus that implements machine-triggered and third-party triggers at scale
 - GitHub Actions documentation — examples of third-party-triggered and chained processing in CI/CD pipelines
 - Any background job library's trigger documentation (Celery, Sidekiq, BullMQ) — all organize their trigger types around the same taxonomy
- Encouragement: the trigger taxonomy is one of those mental models that becomes immediately visible everywhere once you know it. The next time you use a payment gateway, a deployment pipeline, or a scheduling tool, you'll recognize which trigger type it implements — and you'll already know how to design around it reliably.

Note: This is conclusion only — no theory, exercises, or assessment.